

Greenpower Telemetry System

A framework overview for the next team — how the pieces fit together, and why, without the recipe.

This document exists to pass on the *shape* of a working system, not the implementation of one. It deliberately leaves out source code, pin assignments, wiring diagrams, protocol byte layouts, and library-level how-to — not as an oversight, but because figuring those parts out is most of what makes the knowledge stick. Treat this as a map of the territory, not a set of directions.

If a future team rebuilds something like this from scratch using only this document, they will make different (and probably better) choices for the details. That's the point.

1. What the System Does

A race vehicle generates information about itself constantly — how fast it's going, how hard the motor is working, how hot things are getting, where it is on track, how it's oriented and accelerating, and whether its own control electronics are healthy. None of that is useful sitting on the car. The purpose of this system is to get that information off the vehicle, to the people who need to see it, in close to real time, while also keeping a permanent record of every run for later review.

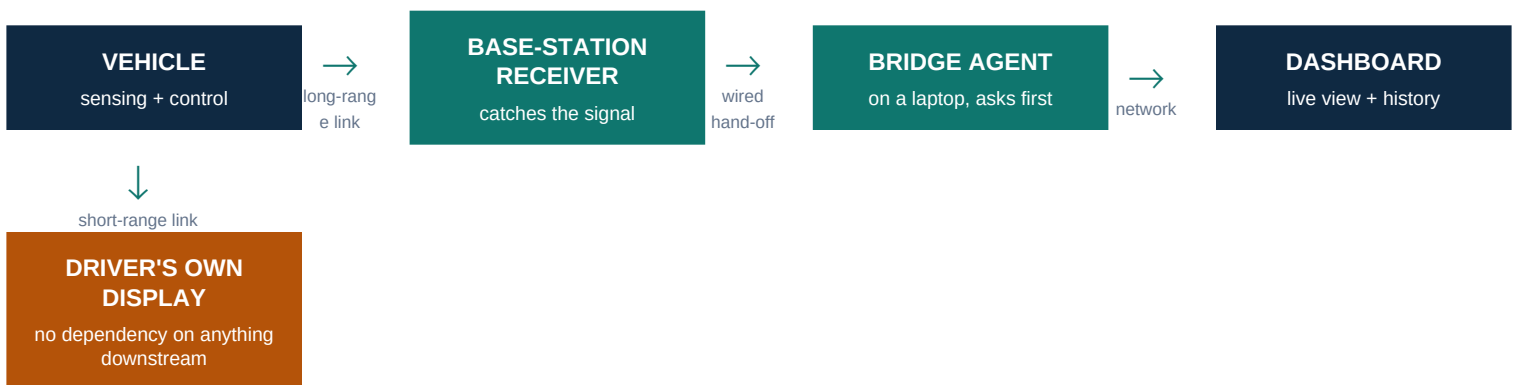
That splits into two different problems that are easy to conflate but worth keeping separate in your head:

- **Getting data off the car at all** — a communications problem, constrained by range, reliability, and the fact that a moving vehicle can't be tethered to anything.
- **Doing something useful with the data once it arrives** — a presentation and storage problem, largely independent of how the data got there.

Every subsystem in this project exists to solve one piece of one of those two problems. Understanding *which* piece each subsystem solves is more valuable than memorizing what each one is called.

2. The Shape of the System

At the highest level, information flows one direction — off the vehicle and outward — through a chain of independent links. Each link only has to trust the one immediately before it; none of them needs to understand the whole chain.



One outbound flow, two independent destinations. The vehicle transmits twice, over two different kinds of link, to two audiences with very different needs.

A separate, smaller controller inside the vehicle box (the motor/throttle controller) reports into the vehicle's own sensing system rather than transmitting on its own — more on why in Section 3.2.

"Everything downstream should be able to fail without taking anything upstream down with it."

That single idea shows up everywhere in this design: the driver's display doesn't need the internet; the dashboard doesn't need to be online for the vehicle to keep transmitting; the base station doesn't need to

understand what the numbers mean, only how to relay them. Keep that principle in mind as you read the rest of this document — it explains more of the "why" than any individual component description does.

3. The Six Pieces

Each piece below is described by its **role**, the **problem it exists to solve**, and the **guarantee it has to uphold** for the rest of the system to work. None of that requires knowing what language it's written in or how its wires are run.

3.1 Vehicle Data-Collection System

Role: the source of truth. Everything else in this system exists to move, store, or display what this piece produces.

The problem it solves: a moving vehicle has a handful of independent sensors measuring physically different things (speed, temperature, orientation, electrical load, position) on different timescales, and none of that is useful until it's gathered into one coherent, regularly-updating snapshot of "what the car is doing right now." This piece is the one place responsible for assembling that snapshot and getting it onto the two outbound radio links.

What it has to guarantee: a consistent, predictable snapshot on a steady cadence, sent out whether or not anything downstream is listening. It never waits for an acknowledgment from the receiver, the dashboard, or anyone else — broadcasting only, no handshake. That's what lets everything downstream be optional.

Built with: an ESP32-S3 dev board with LoRa radio hardware built directly onto it, rather than a plain microcontroller plus a separate LoRa module — one less physical link to wire up and get wrong. Analog readings (battery/motor voltage, current) go through a dedicated external 16-bit ADC chip over I2C instead of the microcontroller's own built-in ADC, which measurably improved accuracy once compared side-by-side — the built-in ADC on this class of chip is a known weak point for precision analog work. The temperature probe is a digital sensor, not an analog thermistor, for the same reason: nothing left to calibrate or drift. The radio link itself runs through an actively-maintained third-party radio library rather than hand-rolled register-level code — getting a long-range radio's timing/modulation parameters right from scratch would have been its own multi-week project on top of everything else. A small local SD card and real-time clock keep an independent on-vehicle backup log, so a dropped radio link during a run doesn't mean lost data.

3.2 Motor / ESC Controller

Role: a second, smaller brain, physically separate from the main data-collection system, responsible for the motor and throttle.

The problem it solves: motor control is a real-time, safety-relevant job on its own — it shouldn't have to share a processor, or compete for attention, with the job of reading a dozen unrelated sensors and formatting radio packets. Splitting it out means a change to one doesn't risk destabilizing the other, and the motor controller can keep doing its job even if something on the sensing side hangs.

What it has to guarantee: it reports its own state (and only its own state) inward to the main data-collection system on a short, local connection — it never talks to the receiver, the agent, or the dashboard directly. From the rest of the system's point of view, the ESC's data simply appears as part of

the vehicle snapshot; nothing further downstream needs to know two separate processors were involved at all.

Built with: a plain, inexpensive general-purpose ESP32 development board — no on-board radio needed, since this piece never transmits wirelessly at all (see its guarantee, above). Motor PWM output runs through the chip's own dedicated hardware PWM peripheral rather than a software-timed loop, which matters for a signal driving real motor hardware — hardware PWM doesn't suffer the small timing jitter a software timing loop is prone to under load. A small local OLED status screen gives on-car diagnostic feedback with zero dependency on anything else in the system. Deliberately the simplest, most generic hardware choice of any piece here — motor control doesn't need the specialized radio hardware the sensing board carries.

3.3 Base-Station Receiver

Role: the fixed point on the ground that catches what the vehicle transmits over its long-range link.

The problem it solves: long-range wireless links and general-purpose computers (laptops) don't talk to each other directly — something has to sit at the radio end and hand packets off over a connection an ordinary computer understands. This piece is deliberately "dumb": it doesn't interpret the data, store it, or decide what to do with it. It just catches and relays, as literally as possible.

What it has to guarantee: whatever it receives gets handed off faithfully and promptly, in a form a laptop can consume without any special hardware drivers. Keeping this piece dumb is what makes it swappable and easy to reason about — interpretation happens later, in one place, not here.

Built with: the exact same radio-capable ESP32-S3 board family, running the same radio library, as the vehicle's own sender. That wasn't a coincidence — two ends of a radio link have to agree closely on hardware and radio settings, and matched hardware on both ends removed an entire category of "is this a settings mismatch or a hardware difference" debugging that otherwise would have eaten real time. Otherwise this board carries nothing extra: no sensors, no display, no unnecessary libraries — kept deliberately minimal to match its "dumb relay" role.

3.4 Bridge Agent

Role: a small background program on whatever laptop the receiver happens to be plugged into. It's the seam between "hardware near the car" and "a service on the internet."

The problem it solves: the receiver only knows how to talk over a wire; the dashboard only knows how to talk over a network. Something has to sit in between, notice when a receiver is actually present, and carry data from one world to the other. It also solves a second, less technical problem: nobody should be broadcasting live data to a public dashboard without knowing it's happening.

What it has to guarantee: it asks a real person before it starts forwarding anything, every single time a receiver connects — unplugging and reconnecting always resets that question. It never assumes yesterday's "yes" still applies today. This is a deliberate human-in-the-loop checkpoint, not an inconvenience to engineer around.

Built with: Node.js, chosen because this piece only needs to run on an ordinary laptop (not be embedded on a chip), and its ecosystem has small, well-maintained packages for exactly the handful of things this piece needs: watching for a USB serial device to appear, reading from it, and firing a real OS-level notification — the actual mechanism behind the "ask before forwarding" consent prompt. Packaged as a genuine Windows installer built with an open-source installer-authoring toolkit, rather than a zip file or a script someone has to run by hand. Worth calling out: an early version used a more common installer format that turned out to be silently blocked by Windows' own built-in application-control policy on corporate/managed laptops; switching to a different, still entirely standard, installer package format sidestepped that block completely — the kind of practical, environment-specific snag you generally only discover by actually testing on a real target machine, not by reading documentation. The small always-on tray icon and status window reuse tools already built into Windows rather than pulling in a heavyweight app-packaging framework, keeping the whole install small and fast.

3.5 Web Dashboard

Role: where humans actually look at the data — live, and after the fact.

The problem it solves: a live number on a screen and a searchable history of every run are two different problems wearing the same trenchcoat. Live data needs to feel instantaneous and doesn't need to be perfect or complete; historical data needs to be durable, complete, and organized into sessions someone can come back to weeks later. This piece has to do both without one job compromising the other.

What it has to guarantee: a clear, honest distinction between "live and current" and "offline" — it should never show stale numbers as if they were fresh. Everything that arrives also gets durably recorded, independent of whether anyone is watching the live view at that moment, so a run can be reviewed or exported later even if nobody was staring at the screen during it.

Built with: a Node.js backend on a minimal, unopinionated web framework, with a small managed Postgres database for durable storage. Telemetry rows are stored as flexible, semi-structured records rather than one rigid database column per sensor reading — a deliberate choice, made after the sensor field list had already changed shape more than once; a rigid schema would need a database migration every single time a field is added or reworked, while a flexible one doesn't care. The frontend is plain, framework-free HTML/CSS/JS with no build step at all — the whole site deploys as literal static files, which kept hosting and deployment almost trivially simple while the UI's actual complexity was still modest enough not to need more. A charting library drives the live graphs and a mapping/tiles library drives the live GPS raceline overlaid on satellite imagery, rather than either being built from scratch. It's also installable as a home-screen mobile app (a Progressive Web App) via a small manifest file and a minimal background worker that deliberately never caches live data — showing a cached, stale reading as if it were live would defeat the entire purpose of a live telemetry tool, so "installable app feel" and "cached offline data" were kept strictly separate. Hosting is a managed application platform (Railway) that runs the app continuously and attaches a managed database with very little manual server administration, rather than self-hosting a machine someone has to keep patched and running. The public domain name is registered through a standard domain registrar (GoDaddy), with DNS and HTTPS handled through Cloudflare sitting in front of the hosted app — a common, low-effort way to get a stable public URL, automatic SSL, and a layer of traffic protection without operating any of that infrastructure directly.

3.6 Driver's Own Display

Role: the one audience that cannot be allowed to depend on anything outside the car — not the receiver, not the agent, not the internet, not even whether anyone is watching the dashboard at all.

The problem it solves: the driver needs certain information *right now*, regardless of whether the base station is in range, a laptop is plugged in, or the dashboard is even reachable. That means it can't be a client of anything downstream in the chain — it has to get its information straight from the vehicle over its own short-range link, completely independent of the long-range path everything else relies on.

What it has to guarantee: it keeps working when everything else in this document has failed or simply isn't present. That's the entire reason it exists as a separate piece instead of just being "the dashboard, but in the car."

Built with: a compact ESP32-S3 board paired with a 4.2-inch reflective LCD panel from the same manufacturer as the board. The display technology itself was a real, considered choice among three real options: a backlit TFT screen (bright and fast, but washes out and becomes unreadable in direct outdoor sunlight — a real problem for a vehicle raced outdoors), true e-paper (perfectly sunlight-readable, but far too slow to redraw for a live, several-times-a-second dashboard — e-paper is built for mostly-static content, like an e-reader page), or a reflective LCD (sunlight-readable like e-paper, since it reflects ambient light rather than emitting its own, but fast enough to redraw for genuinely live data). The reflective LCD was the only one of the three that satisfied both real constraints — sunlight legibility and live refresh speed — at once. Drawing on the panel goes through the panel manufacturer's own low-level display driver, with a widely-used, well-documented open-source graphics library layered on top for shapes, text, and an off-screen drawing buffer, rather than writing raw pixel-pushing logic from nothing. It talks to the vehicle over the same short-range, peer-to-peer wireless technology built into the board — not the long-range link the base station uses — well matched to "one sender, one receiver, a few feet apart, both riding on the same vehicle," with none of the pairing/infrastructure overhead a longer-range link would need for that short a hop.

4. A Note on Testability

None of the downstream pieces — the receiver, the agent, the dashboard, the driver's display — actually need the real vehicle to be useful during development. If a downstream piece only cares about receiving a well-formed snapshot on schedule, anything that produces a well-formed snapshot on schedule is indistinguishable to it from the real vehicle.

That observation is worth internalizing as a general design habit, not just a fact about this particular project: when a piece of a system is expensive, slow, or physically inconvenient to involve in everyday testing (a race car is a good example), it's worth asking what the *minimum contract* that piece has to fulfill is, and building something small that fulfills just that contract. Everything downstream keeps working, keeps getting tested, and keeps improving without ever needing the real thing plugged in.

The dashboard also has its own equivalent trick built in for exercising the display and layout without needing even a stand-in transmitter running. The general principle is the same one twice, at two different points in the chain.

5. Cross-Cutting Principles Worth Carrying Forward

Broadcast, don't handshake.

The vehicle never waits to hear back from anything downstream before it keeps transmitting. A design where the source depends on an acknowledgment from a consumer is a design where a consumer problem becomes a source problem.

Keep relays dumb.

The piece physically closest to the radio hardware does the least interpretation. Understanding what the data *means* is concentrated in as few places as possible, so there's only one place to get it right (and one place to fix it, if it's ever wrong).

Never let downstream silence look like upstream failure, or vice versa.

A dashboard showing "offline" should mean the data genuinely stopped arriving, not that a webpage lost its network connection. Conflating those two situations is one of the easiest ways to make a live system feel untrustworthy.

Ask before you broadcast.

Anything that turns "data reached a laptop" into "data is now visible to the public internet" is a decision, not a default. A system that quietly starts forwarding on its own is a system nobody fully trusts.

Design the minimum contract, then build a stand-in that meets only it.

Real hardware is slow to involve in day-to-day development. Once you know exactly what a downstream piece actually needs to receive, you can build something far simpler than the real source that gives it exactly that — and test everything downstream freely.

Separate what must survive from what's merely convenient.

The driver's own display exists because "nice to have live telemetry in the pits" and "the driver needs to see this number regardless of circumstances" are different tiers of requirement, and collapsing them into one dependency chain would make the important one only as reliable as the least important link in it.

6. What This Document Leaves Out, On Purpose

You will not find, in this document: what hardware was used, how anything is wired, what the data actually looks like on the wire, what any function or file is called, or what libraries or services anything is built on. That's deliberate, not an oversight.

Every one of those specifics is something you can work out yourself, and you'll understand it far better having worked it out than having been handed it. What matters — and what doesn't decay the moment a library changes or a part gets swapped — is the shape described in Sections 2 and 3, and the reasoning in Section 5. Hold onto those, and the details are yours to rebuild, improve, or do differently.

Good luck. Figure out the rest yourself — that's how the first version got built too.